MK61s F401/F411 + APP в Arduino IDE

Версия документа: 04.11.2026

Готовые комплекты STM32 доступны в GitHub Releases. Ниже описана локальная сборка через Arduino IDE.

Эта инструкция собирает через обычные кнопки Arduino IDE согласованный комплект для STM32F401CC или STM32F411CE. Устанавливаются две отдельные платы с одним набором меню: `MK61s F401 + APP` и `MK61s F411 + APP`. Ниже показан полный пример F401 mini с включёнными USB-экраном и всеми опциональными компонентами в режиме APP:

```
.build/firmware/mk61s-M-mini-v3-lcd1602-a00-f401/
├── mk61s-M-mini-v3-lcd1602-a00-f401.bin
├── build.flags
├── build.apps
└── System/
    ├── FOCAL.APP
    ├── BASIC.APP
    ├── MARKDOWN.APP
    ├── CHIP8.APP
    ├── SETUP.APP
    ├── USBDISK.APP
    ├── HELP0.TXT
    └── HELP1.TXT
```

Это штатные FOCAL, TinyBASIC, просмотрщики WBMP и Markdown и консоль CHIP-8. Создавать пользовательский `HELL0`, manifest `app.mk61` или менять прошивку для этого не нужно.

Зависимости

Нужны Arduino IDE и пакет платы MK61s, установленный командой из следующего раздела. Начиная с версии 2.0.0 он использует наш нативный runtime и SDK из репозитория. STM32 Core, LiquidCrystal и STM32duino RTC не требуются; установленная версия STM32 Core на эти платы не влияет. Меню, полный набор APP, поточный VM и проверка CRC сохраняются.

Установщик автоматически загружает отдельный GNU Arm GCC `14.2.1-1.1` и проверяет SHA256 по закреплённому списку. При первой установке нужен доступ к GitHub. Можно передать уже установленный компилятор: `-ToolchainDirectory` на Windows или `--toolchain` на macOS/Linux; аргумент указывает на каталог `bin` либо его родителя. Поддерживаются также переменные `MK61_ARDUINO_TOOLCHAIN` и `MK61_GCC_TOOLCHAIN`.

Отдельный `arduino-cli` не требуется. На Windows сборка и установка `/System` используют штатные PowerShell 5.1 и .NET; Python, host-компилятор C++ и Bash не нужны. На macOS/Linux нужны Python 3 и host-компилятор C++17 для упаковки APP; Arduino компилирует сам sketch тем же независимым GNU Arm GCC.

Для Upload на Windows отдельно установите STM32CubeProgrammer с его USB DFU-драйвером. Загрузчик прямо вызывает `STM32_Programmer_CLI.exe`: он ищется в `PATH` или в стандартном каталоге ST `C:\Program Files\STMicroelectronics\STM32Cube\STM32CubeProgrammer\bin`. Нестандартный путь можно задать переменной `MK61_STM32_PROGRAMMER`. STM32Tools и BusyBox не требуются. На macOS/Linux установите `dfu-util` либо добавьте STM32CubeProgrammer CLI в `PATH`.

Установка платы

Закройте Arduino IDE и из корня проекта выполните на macOS/Linux:

```
./tools/mk61-arduino-board.cmd
```

На Windows из `cmd.exe` или PowerShell:

```
tools\mk61-arduino-board.cmd
```

Это полиглотный launcher: в macOS/Linux он запускает shell-установщик, а в Windows - PowerShell. Он копирует пакет платы с нашим SDK в sketchbook и при необходимости устанавливает закреплённый GNU Arm GCC. Для Arduino IDE 2 в Windows установщик читает фактический `directories.user` из `%USERPROFILE%\.arduinoIDE\arduino-cli.yaml`, поэтому перенесённый sketchbook и каталог Documents под OneDrive не требуют ручной догадки. Arduino IDE 1 такого файла не создаёт: для неё

используется штатный `Documents\Arduino`, а нестандартный sketchbook нужно один раз передать ключом `-Sketchbook`, как показано ниже. Если стандартный `build_cache.path` проходит через имя пользователя с кириллицей или другими не-ASCII символами, установщик записывает в тот же файл отдельный реальный кэш вида `C:\Users\Public\Documents\MK61Arduino\build-cache-...`. Это не короткое имя и не канонизация исходного пути: Arduino собирает во вновь созданном ASCII-каталоге, а sketchbook, исходники и профиль пользователя остаются на своих местах. Так GNU Arm LTO получает доступный путь к `sketch\objs.a`; LTO не отключается. В выводе команды всегда напечатаны выбранные пути и источник sketchbook.

Если sketchbook перенесён из стандартного каталога:

```
./tools/mk61-arduino-board.cmd --sketchbook /path/to/Arduino
```

```
tools\mk61-arduino-board.cmd -Sketchbook D:\Arduino
```

Проверить установку без изменения файлов:

```
./tools/mk61-arduino-board.cmd --check
```

```
tools\mk61-arduino-board.cmd -Check
```

После обновления репозитория запустите установщик ещё раз: установленная плата является копией служебных файлов. Перед установкой закройте все окна Arduino IDE, после неё запустите IDE заново.

Версия пакета видна прямо в названии платы, например `MK61s F401 + APP (2.0.0)` или `MK61s F411 + APP (2.0.0)`. Это версия интеграции с Arduino IDE, а не прошивки или формата APP. `-Check` показывает установленную версию и версию в текущем проекте и сравнивает скопированные файлы. Одного скачивания нового проекта недостаточно: IDE продолжает использовать свою установленную копию.

Пакет 2.0.0 собирает прошивку непосредственно из открытого sketch нашим runtime и драйверами. Заголовки SDK, linker script и параметры особых единиц компиляции общие с нативной сборкой. Внешние аппаратные библиотеки не подключаются; загрузчику APP доступны полные 20 КиБ RAM. После обновления проекта переустановите пакет платы и перезапустите IDE.

Начиная с пакета 1.2.0 перед компиляцией проверяется совпадение его файлов с проектом. При несовпадении сборка просит закрыть IDE, запустить `tools\mk61-arduino-board.cmd` из обновлённой папки проекта и открыть IDE заново. Для более старой установки сначала выполните обновление вручную. Если Arduino останавливается ещё раньше с Неверный FQBN, переустановите пакет той же командой: проверка параметров FQBN происходит раньше, чем запускаются наши проверки сборки.

Сообщения `System APP build: python.exe failed with exit code 1` и `MK61s Arduino board: py.exe failed with exit code 1` приходят из прежней Windows-обёртки, которая запускала `build_system_app_bundle.py`. В актуальном Windows-пути весь комплект собирается через `build-system-app-bundle.ps1`, включая упаковку APP, а загрузку выполняет PowerShell-установщик. Python-сборщики на Windows больше не собирают APP: при вызове старой обёрткой они сразу сообщают команду переустановки пакета платы. После обновления ZIP нужно заменить и установленный пакет платы:

```
.\tools\mk61-arduino-board.cmd
.\tools\mk61-arduino-board.cmd -Check
```

Запускайте команды из новой папки проекта при закрытой Arduino IDE. Установщик также удаляет из Windows-установки старый `seal-firmware-elf.py`; проверка `-Check` отклоняет установку с этим файлом. После открытия IDE в названии выбранной платы должна быть версия `2.0.0`.

`MK61s F401 + APP` и `MK61s F411 + APP` — вручную установленные платы, поэтому их не будет в Boards Manager. Устанавливать там STM32 Core не нужно. Нужную плату ищите по точному имени в Board Selector либо через Tools → Board → Select Other Board and Port.

Выбор профиля

1. Откройте файл `code/code.ino`. Это служебный основной tab Arduino; код прошивки остаётся в соседнем `mk61s-M.ino`.
2. В Board Selector выберите плату строго по установленному контроллеру: `MK61s F401 + APP` для STM32F401CC или `MK61s F411 + APP` для STM32F411CE.

3. В меню Tools выберите платформу: `mini V3`, `mini V2`, `Classic V3`, `Classic V2` или `40th`.
4. Отдельно выберите экран: LCD1602 · CGROM A00, LCD1602 · CGROM A02, OLED1602 · WS0010 FT=10 или `UC1609`.
5. Выберите Встроен, `APP` или Выключен для FOCAL, TinyBASIC, `Markdown + WBMP viewer` и CHIP-8. Отдельный `WBMP viewer` выбирайте только при выключенном Markdown.
6. При необходимости задайте USB-экран, быстрый Explorer и математический backend. Ручная геометрия графического шрифта остаётся расширенной опцией `mk61-firmware.cmd`; Arduino IDE использует проверенные пресеты. На F411 также можно оставить USB-диск встроенным (штатно) либо выбрать `USBDISK.APP`.

На F411 FOCAL и TinyBASIC по умолчанию являются APP, а Markdown, SETUP, Проводник и USB-диск встроены. Режим `BINARY64` использует нашу `compact_math`. На F401 крупные компоненты по умолчанию вынесены в APP, USB-диск обязательно остаётся `USBDISK.APP`, а математика использует компактный CORE. Любой доступный компонент можно переопределить через те же меню, если выбранная комбинация помещается во Flash.

CHIP-8 по умолчанию выключен. Пользовательские APP всегда поддерживаются единым загрузчиком и отдельного разрешающего переключателя не имеют. WBMP-картинки и CHIP-8 отображаются только на графическом UC1609 либо вместе с пунктом USB-экран. В mini-профиле с LCD1602 и включённым USB-экраном файлы запускаются лишь во время активной desktop-сессии. Если сессии нет или она исчезла, устройство показывает ГРАФИКА / НЕДОСТУПНА.

Допустимы только реальные сочетания:

Платформа	Экран
mini V3	LCD1602 A00/A02 или OLED1602 WS0010
mini V2	LCD1602 A00 или A02; WS0010 только экспериментальной ручной сборкой
Classic V2/V3	UC1609
40th	UC1609

Несовместимая пара останавливает сборку до запуска компилятора с понятной ошибкой. Платформа и экран намеренно остаются двумя независимыми меню.

Меню «Математика» предлагает два режима. `CORE` — самый компактный штатный выбор F401 с точностью ядра МК-61; `BINARY64` сохраняет double-точность, но занимает больше всего Flash. Для тесных Classic V2/UC1609 выбирайте `CORE`.

Прежний пункт `libm` переименован в `BINARY64`: заново выберите его в IDE либо используйте `mk61_math=binary64` в FQBN. Числовое значение `MK61_MATH_BACKEND=0` не изменилось. В самостоятельной GCC-сборке этот же режим использует наше `compact_math`, без внешней libm.

Прежний экспериментальный `CORE + APP FLOAT` несовместим с общей языковой VM: языковые APP используют математику resident. В пакете `2.0.0` сохранённый выбор `mk61_math=core_app_float` автоматически соответствует `CORE` и виден как «CORE (старое значение “CORE + APP FLOAT”)». Можно заново выбрать обычный `CORE`. FOCAL, TinyBASIC и их математические операции остаются доступны.

WS0010-профиль предназначен для Winstar WEH001602A с интерфейсом 6800, питанием 5 В и English/Russian `FT=10`; он не является синонимом A00/A02. Подключение и обязательная аппаратная приёмка описаны в `MK61s-mini-WS0010.md`.

Verify, Export и Upload

Обычный Verify выполняет всю сборку и создаёт каталог в `.build/firmware/`. Имя каталога содержит выбранные платформу и экран. В нём лежат:

- `.bin` resident-прошивки;
- `.elf` той же resident-прошивки с адресами секций и уже записанной post-link CRC;

- только выбранные именно как APP `System/FOCAL.APP`, `System/BASIC.APP`, `System/WBMP.APP`, `System/MARKDOWN.APP`, `System/CHIP8.APP`, `System/SETUP.APP`, `System/USBDISK.APP` и `System/EXPLORER.APP`;
- обязательные для обеих плат `System/HELP0.TXT` и `System/HELP1.TXT`; штатная F411-сборка не создаёт System APP, потому что все включённые по умолчанию компоненты встроены;
- `build.flags` с точными compile-time ключами.

Используйте `.bin` внутри этого каталога вместе с `System` той же сборки. Arduino IDE сохраняет локальный комплект в `.build/firmware/` после Verify или Upload. Итоговый путь выводится в журнале сборки.

`WBMP.APP` и `MARKDOWN.APP` вместе не создаются. Графический `MARKDOWN.APP` открывает и Markdown `T2`, и WBMP `I1`; отдельный `WBMP.APP` нужен только при выключенном Markdown.

Повторная сборка того же профиля удаляет из результата канонический System APP, если компонент выключен или встроен. Другие каталоги и пользовательские APP сборщик платы не трогает.

На Windows кнопка Upload собирает комплект, прошивает resident `.bin` через DFU, ждёт возвращения последовательного порта и через терминальный протокол MKC устанавливает и проверяет файлы этого же комплекта в `/System`. Отдельно открывать USB-диск не нужно. На F401 это работает и сразу после форматирования C6, когда `USBDISK.APP` ещё отсутствует; на F411 используется встроенный USB-диск. Выбранные в IDE APP загружаются, выключенные штатные APP удаляются; пользовательские файлы не трогаются. Плата использует собственный uploader `mk61Upload`, поэтому наследуемый рецепт STM32CubeProgrammer не может незаметно обойти второй этап. Windows-сборка кладёт `mk61-system-installer.ps1` рядом с временным каталогом `System`, и Upload запускает эту копию. Поэтому второй этап не зависит от `build.source.path`, который IDE может разрешить от `C:\Program Files\Arduino IDE`. Начиная с пакета 1.2.3 перед DFU проверяется весь комплект: resident BIN/ELF, установщик и каждый файл `System` должны совпадать с манифестом текущей сборки. Неполный или изменённый комплект нужно пересобрать; до этой проверки прошивка устройства не начинается. Вложенные файлы выбранных модулей проверяются и устанавливаются тем же способом. Штатный поточный VM не требует VMSTORE.APP. Загрузчик не использует свойства внешних платформ Arduino: resident передаётся непосредственно STM32CubeProgrammer. Даже если STM32 Core не установлен, после успешного DFU выполняется тот же второй этап через CDC. После обновления исходников повторно установите плату командой `tools\mk61-arduino-board.cmd` и перезапустите IDE: старая копия `platform.txt` в sketchbook сама не обновится. Команда с `-Check` не ограничивается наличием файлов: она сравнивает установленный пакет с текущим репозиторием и завершится ошибкой `installed but stale`, если IDE всё ещё получила бы старый uploader. Обычная установка заканчивается строкой `Verified uploader: mk61Upload (standalone DFU + automatic /System install).`

Перед Upload переведите BlackPill в системный DFU. Если к компьютеру подключено несколько MK61s, выберите в IDE последовательный COM-порт нужной платы; после прошивки проверяются профиль и номер сборки, полученные от устройства. При повторном открытии порта также сверяется ID калькулятора. Перед Upload закройте все программы, использующие COM-порт этого устройства. После DFU загрузчик до 45 секунд ждёт возвращения CDC и эксклюзивного доступа к выбранному порту; если другая программа продолжает его удерживать, установка завершается с прямым сообщением о занятом порте. Прочие последовательные устройства отключать не требуется, если в IDE выбран правильный COM-порт; без выбранного порта должен однозначно находиться один MK61s. Пакет 1.2.3 разрешает запуск Upload без COM-порта, в том числе при первом подключении контроллера в DFU; после прошивки установщик сам ждёт появления CDC. При ошибке IDE покажет неуспешную загрузку, даже если resident уже прошился; повторно прошивать его для установки APP не надо.

Восстановление без DFU для F401 Classic V2/UC1609 (путь замените на каталог `System` своей сборки, а COM9 — на порт своего устройства):

```
tools\mkc.cmd --install-system ".build\firmware\mk61s-M-classic-v2-uc1609-f401\System" --
expect-profile classic-v2-uc1609 --port COM9
```

Для штатной F411-сборки со встроенным USB-диском добавляется явный ключ, разрешающий отсутствие `USBDISK.APP` и удаляющий его старую внешнюю копию:

```
tools\mkc.cmd --install-system ".build\firmware\mk61s-M-mini-v2-lcd1602-a00-f411\System" --
expect-profile mini-v2-a00 --resident-usbdisk --port COM9
```

При ошибке штатного Upload используйте напечатанную им команду восстановления: она дополнительно содержит `--expect-build` с номером именно этой сборки.

В CI на Windows PowerShell 5.1 собираются настоящие F401 и F411 комплекты, после чего вызывается `arduino-cli upload` через те же рецепты, что использует IDE. Проверяются DFU, появление и занятость COM-порта, identity, все файлы `/System`, CRC передачи и чтение обратно, а также восстановление без DFU. Отдельные сценарии проверяют чужой профиль/сборку, смену устройства, несколько портов, повреждённый комплект, обрыв связи, неверный ACK и повреждение чтения. Программатор и физический COM-порт в этом тесте моделируются; работу USB-драйвера и реального контроллера нужно дополнительно проверять на Windows с платой.

На macOS/Linux Upload пока прошивает только resident. Там по-прежнему нужно скопировать `System` из комплекта в `/System` через USB-диск или МКС. При

Ручная загрузка через STM32CubeProgrammer

Для ручного восстановления в STM32CubeProgrammer предпочтителен подходящий устройству `.elf` из GitHub Release либо свежего каталога .build/firmware/<профиль>/: ELF сам содержит адреса загружаемых секций. Post-link этап переносит в него ту же CRC, что и в BIN, затем повторно преобразует ELF в плоский образ и требует полного побайтового совпадения с запечатанным BIN.

BIN сохраняется для автоматических загрузчиков и ручного резерва. При его выборе укажите адрес `0x08000000`. После записи выполните `Run after programming`/Start либо аппаратный Reset при отпущенном BOOT0. Надпись `DFU flash mode!` после записи может быть старым текстом, оставшимся в памяти LCD, и сама по себе не означает, что новая прошивка запустилась.

Если прошивка пищит и реагирует на клавиши, но LCD1602 показывает сплошные прямоугольники, процессорная часть уже работает, а дисплей не был инициализирован. Проверьте платформу mini V2/V3 и тип WS0010 против LCD1602; A00/A02 меняют набор символов, но не электрическую инициализацию. Даже без экрана точный запущенный профиль можно получить через МКС-командами `identity` и `display status`.

При первом переходе на ABI 6 обновите и resident, и файлы `/System`.

Самостоятельные APP и сжатие

Legacy-путь `tools/build_f401_bundle.sh` запускает отдельные Arduino-сборки с LTO и host-упаковщик ZX0. Он оставлен для пользовательских manifest APP, где пока нужны `arduino-cli`, Bash и отдельный C++17-компилятор.

Платы `MK61s F401 + APP` и `MK61s F411 + APP` вызывают тот же самостоятельный SDK ABI 6, что и командные сборщики. Каждый System APP связывается для виртуальной базы `0x20000000`, без импорта адресов из resident ELF. Загрузчик арендует его размер и исправляет указатели по таблице релокаций. Упаковщик выбирает меньший из ZX0 и ARM Thumb BCJ + ZX0. Все штатные образы вместе с BSS по отдельности укладываются в 20 КиБ; одновременно загружен один APP. Он получает блок своего размера с округлением до 32 байт сверху свободной RAM, ниже защиты стека. Постоянного 20-КиБ резерва нет. APP ABI 2/3/4 нужно пересобрать; готовые ABI 6 продолжают работать без изменений.

Канонический путь `tools/mk61-firmware.cmd` на Windows и macOS/Linux вызывает `tools/build-gcc.cmd` и собирает все включённые модули через общий SDK с тем же выбором сжатия. SETUP содержит экраны платы, RTC и шрифта. Справка терминала извлекается из секции ELF, которая не занимает Flash устройства; копируйте весь `System`. Написание собственных программ на C/C++ описано в отдельном руководстве, изменение размеров модулей — в отчёте.

Диагностика

Сообщение	Что проверить
Плата не видна	Закрыть все окна IDE, повторить установщик и сверить напечатанный sketchbook с File → Preferences → Sketchbook location; после запуска искать `MK61s F401 + APP` или `MK61s F411 + APP` в Board Selector/Tools → Board, а не в Boards Manager.
`Invalid FQBN` / недопустимый параметр 'mk61_font_settings'	Arduino IDE сохранила в FQBN удалённую настройку старой версии платы. Закрыть все окна IDE, обновить репозиторий, повторно выполнить `tools\mk61-arduino-board.cmd` и запустить IDE. Установщик добавит безвредную совместимость со старыми `disabled`/`enabled`; сбрасывать все настройки IDE не нужно.
`LiquidCrystal.h: No such file` / не найден `STM32duino RTC`	Выбрана старая плата или её установленная копия. Обновить проект, закрыть IDE, повторить установщик и выбрать MK61s F401/F411 + APP версии 2.0.0.
`language APPs use the resident runtime; local libm is disabled`	Выберите «Математика → CORE». Обновите проект, закройте IDE, повторно выполните `tools\mk61-arduino-board.cmd` и перезапустите IDE. Пакет `2.0.0` исправляет старый сохранённый выбор автоматически.
`STM32 core ... is selected; MK61s requires 2.12.0`	Сработал устаревший запрет установленной копии пакета. Закройте IDE, обновите проект, повторите `tools\mk61-arduino-board.cmd` и перезапустите IDE. Понижать версию Core не нужно: пакет 2.0.0 от него не зависит.
`lto-wrapper ... serial compilation` / `missing .note.GNU-stack`	Это предупреждения; итоговую причину остановки ищите дальше в логе. Созданный BIN и успешная CRC подтверждают только готовность resident, весь комплект должен закончиться сообщением `unified ABI 6 bundle built by Arduino IDE`.
`read ECONNRESET`	Соединение IDE оборвалось, но эта строка не объясняет причину. Закройте IDE, обновите проект, повторите `tools\mk61-arduino-board.cmd`, затем `-Check` и запустите IDE заново. При повторении запустите IDE из PowerShell и сохраните её вывод со строками `daemon`, `panic` и `Daemon exited`; отдельно укажите версию IDE. Один готовый BIN не подтверждает сборку `/System`.
`incompatible platform/display pair`	Выбрать LCD для mini либо UC1609 для Classic/40th.
OLED включается, но текст неверный	Выбрать OLED1602 · WS0010 FT=10, затем проверить `display test map 0..7`.
`WBMP/CHIP-8 requires UC1609 or USB Screen`	Включить USB-экран для mini либо выключить графические APP.
`exceeds the 20 KiB APP image limit`	Выбранный APP вырос сверх лимита одного образа; это не постоянный резерв SRAM и не ошибка DFU.
`app/firmware mismatch`	Старый ABI 2/3/4 нужно пересобрать; для ABI 6 проверьте версию публичных сервисов.

Сообщение	Что проверить
`python.exe`, `python3.exe` или `py.exe failed with exit code ...`	Arduino запускает старую установленную копию пакета платы. Закройте IDE, обновите `main`, из новой папки выполните `tools\mk61-arduino-board.cmd`, затем `tools\mk61-arduino-board.cmd -Check` и перезапустите IDE. Актуальная Windows-сборка и Upload используют PowerShell/.NET.
`C++17 compiler not found`	Установлен старый пакет платы или старые скрипты репозитория. В свежей Windows-сборке APP пакуются PowerShell/.NET-кодом и Visual Studio/LLVM не требуются; обновите `main`, повторно выполните `tools\mk61-arduino-board.cmd` и перезапустите IDE.
`lto1.exe ... sketch\objs.a failed`	Путь сборочного кэша IDE содержит символы, потерянные Windows-версией GNU Arm LTO. Закройте IDE, повторно выполните `tools\mk61-arduino-board.cmd` и перезапустите IDE. Установщик создаст отдельный ASCII-кэш, сохранив LTO.
`STM32CubeProgrammer CLI not found`	Установите STM32CubeProgrammer с USB DFU-драйвером. Для нестандартной установки задайте полный путь к CLI через `MK61_STM32_PROGRAMMER`. STM32 Core и BusyBox не нужны.
`Error missing binary file argument!`	Используется старая копия MK61 uploader, которая передавала Windows-путь STM32 shell-обёртке отдельными короткими аргументами. Обновите `main`, закройте IDE, повторно выполните `tools\mk61-arduino-board.cmd`, перезапустите IDE и повторите Upload.
Upload не находит устройство	Проверить, что выбрана плата для реально установленного F401/F411, перевести контроллер в системный DFU и повторить Upload.
`MKC terminal installer not found: C:\Program Files\Arduino IDE\tools\.mkc\mkc.ps1`	Используется старый uploader, который ошибочно полагается на `build.source.path` во время Upload. Обновите `main`, закройте IDE, повторно выполните `tools\mk61-arduino-board.cmd`, перезапустите IDE и заново соберите проект. Если resident уже прошит, достаточно приведённой выше команды `mkc.cmd --install-system`; повторный DFU не нужен.
`System APP installation failed`	Resident уже может быть прошит. Проверьте COM-порт и закройте все программы, использующие его, затем выполните показанную выше команду `mkc.cmd --install-system` без повторного DFU.

Обновление старых пакетов

При сообщении прошивка mini-v2-a00 не соответствует пакету mini-v2-lcd1602-a00 в пакете платы до `1.2.2` загрузчик смешивал имя каталога сборки и ID прошивки. Закройте IDE, обновите проект, повторно выполните `tools\mk61-arduino-board.cmd`, перезапустите IDE и выберите плату `2.0.0`. Если resident уже прошит, в команде восстановления замените только `--expect-profile mini-v2-lcd1602-a00` на `--expect-profile mini-v2-a00` и укажите настоящий COM-порт. Каталог `System` переименовывать не нужно.

В пакете платы 1.2.4 служебные команды F401 загружаются из `/System/TERMINAL.APP` и `/System/CLOCK.APP`. Они собираются и устанавливаются автоматически вместе с остальными System APP, в Windows — средствами PowerShell. Это освобождает внутреннюю Flash для поточного VM. После обновления проекта повторно запустите установщик платы и выполните обычную загрузку из IDE: комплект `/System` должен соответствовать новой прошивке. Команды `identity`, `fsls`, `fsget`, `fsput` и базовый отчёт `df` доступны до установки `/System`, поэтому восстановление через CDC остаётся возможным.

На F401 USB-диск запускается после завершения графической программы BASIC/FOCAL. Активный рисунок и состояние вложенного `PRINT` сохраняются до `END`; после этого буфер освобождается для импорта файлов. При активной графической сессии запуск диска возвращает ошибку занятости, не стирая рисунок.